OneCycleLR#

https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.OneCycleL

Description:

Sets the learning rate of each parameter group according to the 1cycle learning rate policy. The 1cycle policy anneals the learning rate from an initial learning rate to some maximum learning rate and then from that maximum learning rate to some minimum learning rate much lower than the initial learning rate. This policy was initially described in the paper Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. The 1cycle learning rate policy changes the learning rate after every batch. step should be called after a batch has been used for training. This scheduler is not chainable. Note also that the total number of steps in the cycle can be determined in one of two ways (listed in order of precedence):

Function Signature

```
class torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr,
total_steps=None, epochs=None, steps_per_epoch=None,
pct_start=0.3, anneal_strategy='cos', cycle_momentum=True,
base_momentum=0.85, max_momentum=0.95, div_factor=25.0,
final_div_factor=10000.0, three_phase=False, last_epoch=-1)
[source]#

Parameters

get_last_lr()[source]#

Return type

get_lr()[source]#
```

Returns:

dict[str, Any]

Code Examples

```
>>> data_loader = torch.utils.data.DataLoader(...)
>>> optimizer = torch.optim.SGD(model.parameters(), lr=1e-4,
momentum=0.9)
>>> scheduler = torch.optim.lr_scheduler.OneCycleLR(
...     optimizer, max_lr=0.01,
steps_per_epoch=len(data_loader), epochs=10
... )
>>> for epoch in range(10):
>>>     for batch in data_loader:
>>>         train_batch(...)
>>>         optimizer.step()
>>>         scheduler.step()
```

Detailed Documentation

Documentation

Sets the learning rate of each parameter group according to the 1cycle learning rate policy.

The 1cycle policy anneals the learning rate from an initial learning rate to some maximum learning rate and then from that maximum learning rate to some minimum learning rate much lower than the initial learning rate. This policy was initially described in the paper Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates.

The 1cycle learning rate policy changes the learning rate after every batch. step should

be called after a batch has been used for training.

This scheduler is not chainable.

Note also that the total number of steps in the cycle can be determined in one of two ways (listed in order of precedence):

A value for total_steps is explicitly provided.

A number of epochs (epochs) and a number of steps per epoch (steps_per_epoch) are provided. In this case, the number of total steps is inferred by total_steps = epochs * steps_per_epoch

You must either provide a value for total_steps or provide a value for both epochs and steps_per_epoch.

The default behaviour of this scheduler follows the fastai implementation of 1cycle, which claims that “unpublished work has shown even better results by using only two phases”. To mimic the behaviour of the original paper instead, set three_phase=True.

optimizer (Optimizer) – Wrapped optimizer.

max_lr (float or list) – Upper learning rate boundaries in the cycle for each parameter group.

total_steps (int) – The total number of steps in the cycle. Note that if a value is not provided here, then it must be inferred by providing a value for epochs and steps_per_epoch. Default: None

epochs (int) – The number of epochs to train for. This is used along with steps_per_epoch in order to infer the total number of steps in the cycle if a value for total_steps is not provided. Default: None

steps_per_epoch (int) – The number of steps per epoch to train for. This is used along with epochs in order to infer the total number of steps in the cycle if a value for total_steps is not provided. Default: None

pct_start (float) – The percentage of the cycle (in number of steps) spent increasing the learning rate. Default: 0.3

anneal_strategy (str) – {‘cos’, ‘linear’} Specifies the annealing strategy: “cos” for cosine annealing, “linear” for linear annealing. Default: ‘cos’

cycle_momentum (bool) – If True, momentum is cycled inversely to learning rate between ‘base_momentum’ and ‘max_momentum’. Default: True

base_momentum (float or list) – Lower momentum boundaries in the cycle for each parameter group. Note that momentum is cycled inversely to learning rate; at the peak of a cycle, momentum is ‘base_momentum’ and learning rate is ‘max_lr’. Default: 0.85

max_momentum (float or list) – Upper momentum boundaries in the cycle for each parameter group. Functionally, it defines the cycle amplitude (max_momentum - base_momentum). Note that momentum is cycled inversely to learning rate; at the start of a cycle, momentum is ‘max_momentum’ and learning rate is ‘base_lr’ Default: 0.95

div_factor (float) – Determines the initial learning rate via initial_lr = max_lr/div_factor Default: 25

final_div_factor (float) – Determines the minimum learning rate via min_lr = initial_lr/final_div_factor Default: 1e4

three_phase (bool) – If True, use a third phase of the schedule to annihilate the learning rate according to ‘final_div_factor’ instead of modifying the second phase (the first two phases will be symmetrical about the step indicated by ‘pct_start’).

last_epoch (int) – The index of the last batch. This parameter is used when resuming a training job. Since step() should be invoked after each batch instead of after each epoch, this number represents the total number of batches computed, not the total number of epochs computed. When last_epoch=-1, the schedule is started from the beginning. Default: -1

Return last computed learning rate by current scheduler.

Compute the learning rate of each parameter group.

Load the scheduler's state.

state_dict (dict) – scheduler state. Should be an object returned from a call to state_dict().

Return the state of the scheduler as a dict.

It contains an entry for every variable in self.__dict__ which is not the optimizer.